

FORMA, explicada desde el código.

Aplicación Android nativa de entrenamiento y nutrición personalizada. Este reporte describe el producto, las funciones que lo implementan, la arquitectura, el modelo de datos, el motor de personalización y las pruebas que fijan su comportamiento.

1. Resumen
2. Producto
3. Recorrido y pantallas
4. Motor de personalización
5. Mapa de funciones
6. Arquitectura
7. Datos, nube y privacidad
8. Sistema de diseño
9. Stack y pruebas
10. Decisiones y límites

1. Resumen

FORMA arma una rutina semanal y un plan de comidas a partir de un `UserProfile`: deporte, nivel, objetivo, equipo disponible y alimentos preferidos. Acompaña la sesión con checks y un cronómetro de descanso, permite cerrarla con una foto, ofrece un feed de comunidad con reacciones y un catálogo de artículos, y responde preguntas con un asistente que usa los datos del perfil.

El módulo único :app sigue MVVM con flujo unidireccional. Hay 76 archivos Kotlin y cerca de 13800 líneas. La interfaz está en español. La aplicación arranca en Android 8.0 (API 26), compila con SDK 35 y usa JDK 17.

PIEZA	DÓNDE VIVE	QUÉ GARANTIZA
Rutina	<code>RoutineGenerator.generate</code>	Siete días. En gym, solo equipo marcado. Misma semilla, mismo plan.
Dieta	<code>MealPlanGenerator.optionsFor</code>	Cinco opciones por comida, sesgadas al gusto y a la meta calórica.
Coach	<code>LocalCoach.answer</code>	Dieciséis intenciones. Respuestas con peso, IMC y calorías reales.
Memoria	Room + DataStore	La app funciona sin Firebase.
Nube	<code>FormaCloudStore</code>	Contrato único. NoOp o Firestore/Storage.

2. Producto

Deportes, equipo y alimentos

El enum `Sport` define ocho disciplinas: gym, yoga, running, pilates, ciclismo, natación, CrossFit y boxeo. Cada una trae título y subtítulo propios para la pantalla de equipo.

`EquipmentCatalog.groupsFor` devuelve los grupos de ese deporte. El catálogo suma 117 `EquipmentItem.defaultSelection`, `allIds` y `labelFor` cubren la selección inicial, el “marcar todo” y las etiquetas.

`ExperienceLevel` tiene principiante, intermedio y avanzado. `Goal` tiene bajar grasa (−350 kcal), ganar músculo (+300), mantenerse (0) y más resistencia (+150). Esos enteros se llaman `calorieShift`.

`IngredientCatalog` lista 63 alimentos en siete categorías: proteína, carbohidratos, fibra y verduras, grasas, frutas, lácteos y snacks. `name` resuelve un id a su nombre visible. `defaultSelection` propone un conjunto inicial para el onboarding.

Identidad del atleta

`UserProfile` guarda id, nombre, correo, foto, edad, peso, estatura, deporte, nivel, objetivo, equipo, ingredientes y la bandera de onboarding. Las propiedades calculadas son el contrato nutricional de la app:

- `bmi` — peso dividido entre estatura en metros al cuadrado, redondeado a un decimal.

- `bmiLabel` — Bajo por debajo de 18.5, Normal hasta 25, Alto hasta 30, Muy alto después.
- `firstName` — primera palabra del nombre, o “atleta”.
- `dailyCalories` — Mifflin–St Jeor con factor de actividad y `calorieShift`, redondeado a 10 kcal.
- `proteinTargetG` — 2.0 g/kg para músculo, 2.2 g/kg para bajar grasa, 1.7 g/kg en el resto.

La fórmula calórica, tal como está en el modelo:

```
base = 10·kg + 6.25·cm - 5·edad + 5
actividad = 1.375 | 1.55 | 1.725 según el nivel
kcal = redondear((base · actividad + calorieShift) / 10) · 10
```

Los factores de actividad corresponden a una persona ligeramente activa, moderadamente activa y muy activa. El modelo usa el nivel de entrenamiento como proxy de esa actividad.

3. Recorrido y pantallas

Puerta de entrada

`MainActivity` instala `FormaTheme` y compone `FormaApp`. `RootViewModel` publica un `RootState` sellado: `Loading`, `NeedsAuth`, `NeedsOnboarding` o `Ready`. En pantallas anchas el contenido se limita a 600 dp (`MaxContentWidth`) para que las líneas no se estiren.

`AuthViewModel` alterna entre entrar y crear cuenta (`toggleMode`), recoge nombre, correo y contraseña, y dispara `submit` o `signInWithGoogle`. La validación de la implementación local vive en `LocalAuthRepository`.

El onboarding tiene cinco pasos en `OnboardingUiState.totalSteps`. `canContinue` exige nombre de al menos dos caracteres en el paso 0, equipo no vacío en el paso 3 y al menos tres ingredientes en el paso 4. `next` avanza o, en el último paso, llama `ProfileRepository.save`. `onSport` reinicia el equipo. `selectAllEquipment` y `clearEquipment` operan sobre el deporte actual.

Navegación

El objeto `Routes` declara las rutas. `bottomTabs` construye la barra con iconos outlined y rounded de Material. `FormaBottomBar` solo se muestra cuando la ruta actual es una pestaña, con animación de fade y slide. `navigateToTab` hace `popUpTo(HOME)` con `saveState`, `launchSingleTop` y `restoreState`.

Rutas con argumentos: `recipe/{recipeId}/{dayIndex}/{slotId}`, `article/{articleId}` y `finish/{dayIndex}`. También existen perfil, notificaciones e ingredientes.

Inicio

`HomeViewModel` combina cuatro flujos: `perfil`, `stats()`, `recommended()` y la sesión de hoy. `todayWeekDay` resta 2 al `DAY_OF_WEEK` de `Calendar` para alinear domingo con el índice de `WeekDay`. `greeting` parte el día en 5-11, 12-18 y el resto. `TrainingStats` expone entrenos de la semana, racha, kcal y días completados. `unreadNotifications` es un indicador derivado de si ya hay sesiones guardadas. La pantalla compone saludo, tira semanal, métricas, tarjeta del entreno, IMC, accesos y teasers de artículos (`HomeContent`, `WeekStrip`, `TodayCard`, `ProfileCard`, `ArticleTeaser`).

Rutina

`RoutineViewModel.load` observa el perfil y pide `weeklyRoutine`. El estado `Content` calcula `doneCount`, `progress` y `allDone` con la clave `exerciseKey(day, exerciseId)`, que es "`índice|id`". `selectDay` cambia el día y cancela el timer. `toggleExercise` persiste el check y, si queda completado, llama `startTimer` con los segundos de descanso del ejercicio. El timer es una corrutina que descuenta cada 1000 ms hasta publicar `running = false`. `resetDay` limpia los logs de ese día.

`FinishWorkoutViewModel` recoge foto, pie de foto y la decisión de compartir. `save` construye una `WorkoutSession` y, si corresponde, publica en la comunidad. La UI ofrece `ShareToggle`.

Una `RoutineSession` trae día, título corto, título, foco, ejercicios, minutos estimados, kcal estimadas y `isRest`. Cada `Exercise` tiene id, nombre, músculo, series, repeticiones, descanso, tip y equipo.

Dieta

`DayMealPlan` agrupa cuatro `MealOptions`. Cada opción tiene la receta elegida, las alternativas y una foto de plato opcional. Los totales del día suman kcal y macros de lo seleccionado. `MealSlot` fija hora y fracción calórica: desayuno 7:00 y 0.26, almuerzo 13:00 y 0.36, snack 16:00 y 0.12, cena 19:30 y 0.26.

`DietViewModel` recarga el plan, cambia de día, abre el abanico de alternativas y confirma una receta con `chooseOption`. `RecipeDetailViewModel.onPlatePhoto` y `NutritionRepository.setPlatePhoto` guardan la imagen. `IngredientsViewModel` edita el conjunto de alimentos preferidos después del onboarding. La UI desglosa macros con `MacroColumn` y `MacroTile`, usando lima para proteína, aqua para carbohidratos y naranja para grasa.

Comunidad

`Post` referencia un `Author`, imagen, pie, deporte, tiempo, likes y un mapa de reacciones. `Reaction` enumera fuego, fuerza, aplauso, amor e imparable. No existe un modelo de comentario.

`CommunityRepository.feed` filtra por deporte. `toggleLike`, `react` y `toggleFollow` actualizan el estado local y, si la nube está activa, el engagement. `publish` crea el post. `myPosts` alimenta el perfil. `syncMeWithProfile` copia nombre, foto y deporte al autor propio. `CommunitySeed` inserta 8 autores y 8 publicaciones si el feed está vacío. `relativeTime` traduce el timestamp a una frase relativa en la UI.

Asistente y aprendizaje

`AiViewModel` expone el hilo, el texto en curso y las preguntas sugeridas. `send` delega en `ChatRepository.send`, que persiste el mensaje del usuario, pide `AiRepository.answer` y persiste la respuesta. `clear` vacía el historial.

`Article` tiene categoría, minutos, resumen, secciones, imagen y banderas de destacado y guardado. Las categorías son técnica, nutrición, lesiones, recuperación y mentalidad. Hay 10 artículos en `ArticleCatalog`. `LearnViewModel` filtra, busca y alterna la vista de guardados. `ArticleViewModel.toggleSaved` persiste el estado. Inicio consume `recommended`.

Perfil

`ProfileViewModel` actualiza foto, medidas y objetivo, configura los segundos de descanso (`setRestSeconds`), activa recordatorios y cierra sesión. La pantalla muestra publicaciones propias y un editor corporal con `Stepper`.

4. Motor de personalización

Rutina de gym

`RoutineGenerator.generate` bifurca: si el deporte es gym, construye sesiones con el catálogo de máquinas; si no, delega en `SportSessionTemplates.sessions`. La semilla es:

```
id.hashCode · 31 ⊕ equipo ordenado ⊕ nivel·7 ⊕ objetivo·13
```

Esa semilla, más el índice del día, alimenta `Random` al barajar cada grupo muscular. El mismo perfil ve la misma semana hasta que cambia su id, su equipo, su nivel o su objetivo.

`gymPlan` define el split:

NIVEL	DÍAS DE TRABAJO	EJERCICIOS POR DÍA DE TRABAJO	DESCANSO
Principiante	Full A, cardio, tren superior, tren inferior, core	5, 4, 5, 5, 4	Jueves y domingo
Intermedio	Pecho, espalda, pierna, hombro, brazo, full body	4, 4, 5, 4, 5, 4	Domingo
Avanzado	Empuje, tracción, pierna, hombro, brazo, volumen	6 en cada día de trabajo	Domingo

Si el objetivo es bajar grasa o resistencia, el sábado sustituye un grupo por cardio.

`pickExercises` reparte cupos entre los grupos del día, sin repetir id, y se detiene si una ronda no agrega nada. `GymExerciseCatalog.available` filtra por `requires`. El catálogo tiene 60 ejercicios. Si el filtro deja el grupo vacío, entran las variantes sin equipo.

`toExercise` prescribe así:

- Series: 3 en principiante; 4 o 3 en intermedio según sea compuesto; 5 o 4 en avanzado. El primer ejercicio del día es el compuesto.
- Repeticiones: rango alto del catálogo si el objetivo es grasa o resistencia, o si la persona es principiante. Rango bajo en compuestos de intermedio y avanzado. Rango medio en el resto.
- Descanso: 90/45 s si el objetivo es bajar grasa; 120 s en compuestos; 60 s en principiante; 90 s en el resto.

Los minutos estimados son la suma de series por 3, más 10. Las kcal estimadas son minutos por 8.5. Un día de descanso publica sesión vacía con `isRest`.

Las plantillas de los otros deportes viven en `SportSessionTemplates`, con funciones `running`, `yoga`, `pilates`, `cycling`, `swimming`, `crossfit` y `boxing`, más auxiliares `drill`, `rest` y `volumeFactor`.

Plan de comidas

`RecipeCatalog` reúne 57 recetas entre desayunos, almuerzos, snacks, cenas y opciones más contundentes. Cada `Recipe` trae kcal, proteína, carbohidratos, grasa, minutos, ingredientes con cantidad y pasos. `bySlot` las agrupa. `byId` las busca.

`optionsFor` crea un `Random` con `profile.id.hashCode xor (day.index * 977)`. Para cada slot calcula la meta como `dailyCalories * share` y puntúa cada receta:

```
cobertura = ingredientes favoritos / ingredientes de la receta
penalización = |kcal - meta del slot| / meta del slot
score = cobertura · 1.5 - penalización · 1.3 + jitter
jitter = random.nextDouble() · 0.12
```

Toma las 10 mejores, las rota según el día y se queda con 5. Ordena esas cinco por cercanía calórica para que la primera —la seleccionada por defecto— no deje el día corto. Si el catálogo de un slot tuviera menos de cinco, rellena con el resto del pool sin duplicar.

Coach local y coach remoto

`LocalAiRepository` implementa `AiRepository` con el nombre de proveedor «FORMA local». Inserta una latencia simbólica —220 ms en rutina, 180 ms en comidas, 600 ms en chat— para que los estados de carga de la UI sean reales, y delega en los generadores y en `LocalCoach`.

`LocalCoach.answer` normaliza a minúsculas y quita acentos. Elige la intención cuya lista de palabras aparece más veces. Las dieciséis intenciones cubren proteína, sentadilla, preentreno, descanso, hombro, cardio, IMC, calorías, hidratación, creatina, sueño, agujetas, rutina del día, pérdida de grasa, ganancia muscular y dolor articular. Varias ramifican con `Goal` o con la ausencia de perfil. El fallback saluda por `firstName`, recuerda deporte, nivel y objetivo, y recomienda un artículo elegido con una semilla derivada del hash de la pregunta, de modo que la misma pregunta apunta al mismo artículo.

`suggestedQuestions` ofrece seis arranques que la pantalla de IA muestra como chips.

`RemoteAiRepository` reutiliza el motor local para `generateWeeklyRoutine` y `generateDayMeals`. `answer` llama a `requestCompletion` solo si `BuildConfig.AI_API_KEY` no está vacía. El cliente `OkHttp` espera 15 s de conexión y 45 s de lectura. El cuerpo es JSON de chat completions con temperatura 0.6, un mensaje de sistema construido con el perfil y el historial reciente. Si la llamada lanza, el `runCatching` vuelve a `LocalAiRepository.answer`. Las constantes salen de `local.properties` o del entorno: `FORMA_AI_API_KEY`, `FORMA_AI_BASE_URL` (por defecto la API de OpenAI v1) y `FORMA_AI_MODEL` (por defecto gpt-4o-mini). Activarlo exige cambiar el `@Binds` de `AiModule`.

5. Mapa de funciones

Lista de las funciones y métodos que sostienen el producto, agrupados por el lugar donde se invocan o se implementan.

Dominio — interfaces

CONTRATO	OPERACIONES
AuthRepository	signInWithEmail, signUpWithEmail, signInWithGoogle, signOut, flujo currentUser
ProfileRepository	current, save, updatePhoto, clear, flujo profile
TrainingRepository	weeklyRoutine, setExerciseCompleted, resetDay, finishWorkout, flujos de claves, sesiones y stats
NutritionRepository	dayPlan, chooseOption, setPlatePhoto, ingredientCatalog, choicesChanged
CommunityRepository	feed, myPosts, followingCount, toggleLike, react, toggleFollow, publish, syncMeWithProfile
LearnRepository	articles, article, recommended, toggleSaved
ChatRepository	messages, send, clear, suggestedQuestions
AiRepository	generateWeeklyRoutine, generateDayMeals, answer, providerName

ViewModels

CLASE	FUNCIONES PÚBLICAS
AuthViewModel	toggleMode, onName, onEmail, onPassword, submit, signInWithGoogle
OnboardingViewModel	prefill, onName, onPhoto, onAge, onWeight, onHeight, onGoal, onLevel, onSport, toggleEquipment, selectAllEquipment, clearEquipment, toggleIngredient, back, next
HomeViewModel	retry. Auxiliares de archivo: todayWeekDay, greeting
RoutineViewModel	selectDay, toggleExercise, resetDay, startTimer, stopTimer, retry
FinishWorkoutViewModel	onPhoto, onCaption, toggleShare, save
DietViewModel	reload, selectDay, toggleOptions, chooseOption, retry
RecipeDetailViewModel	onPlatePhoto
IngredientsViewModel	toggle, save

CLASE	FUNCIONES PÚBLICAS
CommunityViewModel	selectSport, toggleLike, openReactions, closeReactions, react, toggleFollow
AiViewModel	onInput, send, clear
LearnViewModel	selectCategory, onQueryChange, toggleOnlySaved, toggleSaved
ArticleViewModel	toggleSaved
ProfileViewModel	updatePhoto, updateBody, updateGoal, setRestSeconds, setReminders, signOut

Generación, catálogos y nube

UNIDAD	FUNCIONES
RoutineGenerator	generate, seedOf, gymSessions, gymPlan, pickExercises, toExercise
MealPlanGenerator	optionsFor. Constantes OPTIONS_PER_MEAL = 5 y pool de 10 candidatas
LocalCoach	answer, normalize, weightOf, fallback
RemoteAiRepository	answer, requestCompletion, buildMessages, systemPrompt
GymExerciseCatalog	available
EquipmentCatalog	groupsFor, allIds, defaultSelection, labelFor
RecipeCatalog	byId, más los constructores internos de receta
ArticleCatalog	byId
IngredientCatalog	name
CommunitySeed	posts
SportSessionTemplates	sessions y una función por deporte
FormaCloudStore	pullUserData, upsertProfile, logs, sesiones, comidas, chat, artículos, publishPost, updatePost, engagement, follow, pullCommunityFeed, uploadUserFile
CloudSyncManager	pullIfNeeded, empuje de foto de perfil y de archivos locales
ImageStore	persist

UNIDAD	FUNCIONES
FormaPreferences	saveUser, clearUser, setRestSeconds, setRemindersEnabled

Sistema de diseño

Los composables reutilizables están en `Components.kt`: `ScreenHeader`, `FormaCard`, `SectionTitle`, `FormaPill`, `FormaPillRow`, `PrimaryButton`, `SecondaryButton`, `StatTile`, `FormaAvatar`, `FormaTextField`, `FormaProgressBar`, `LoadingState`, `ErrorState`, `EmptyState`, `QuickActionCard`, `ListRowChevron`, `ConfirmationDialog` y `TagChip`. `FormaImage` y `artworkFor` resuelven ilustraciones de catálogo y fotos del usuario. `FormaTheme` aplica el esquema oscuro.

6. Arquitectura

Paquetes bajo `com.forma.app`:

- `core` — persistencia de imágenes elegidas por la persona.
- `designsystem` — color, tipo, tema, imagen y componentes.
- `domain.model` y `domain.repository` — modelos sin Android y contratos.
- `data.local` — Room y DataStore.
- `data.catalog` — contenido en español.
- `data.ai` — motores local y remoto.
- `data.repository` — implementaciones Room-first.
- `data.remote` — contrato de nube, NoOp, Firebase y sincronización.
- `di` — módulos Hilt.
- `ui` — una carpeta por superficie, más navegación y `FormaApp`.

El patrón es unidireccional. Un repositorio expone `Flow`. El `ViewModel` lo combina en un `StateFlow` con `stateIn` y `SharingStarted.WhileSubscribed(5_000)`. La pantalla recoge con `collectAsStateWithLifecycle`. Los eventos de UI son funciones del `ViewModel` que lanzan corrutinas en `viewModelScope`. Los estados de carga y error son tipos sellados (`HomeUiState`, `RoutineUiState`) o banderas dentro del `data class`.

Hilt está instalado en `SingletonComponent`. `DatabaseModule` construye `FormaDatabase` con `fallbackToDestructiveMigration` y provee los seis DAO más `FormaPreferences`. `RepositoryModule` ata las seis implementaciones de producto. `AiModule` ata `LocalAiRepository`. `AuthModule` elige `auth` con `Lazy` para no construir Firebase si no hace falta. `CloudModule` elige `NoOpCloudStore` o `FirebaseCloudStore` con el mismo criterio. El

plugin de Google Services solo se aplica si existe `app/google-services.json`; si no, el proyecto compila igual.

`FormaApplication` arranca Hilt. KSP genera Room en Kotlin y escribe el esquema en `app/schemas`.

7. Datos, nube y privacidad

Entidades Room:

ENTIDAD	ROL
<code>UserProfileEntity</code>	Perfil único. <code>toDomain</code> / <code>toEntity</code> traducen al modelo.
<code>ExerciseLogEntity</code>	Check de un ejercicio en un día.
<code>WorkoutSessionEntity</code>	Sesión terminada: duración, kcal, foto, nota.
<code>MealChoiceEntity</code>	Receta elegida y foto de plato por día y slot.
<code>AuthorEntity</code> / <code>PostEntity</code>	Feed, reacciones serializadas y follows.
<code>ChatMessageEntity</code>	Historial del asistente.
<code>ArticleStateEntity</code>	Artículos guardados.

Los DAO correspondientes ofrecen observación con `Flow`, lecturas suspendidas, `upsert` y borrados por día o totales. `Converters` serializa listas de strings. `FormaPreferences` guarda en `DataStore` la sesión de la auth local y los ajustes de descanso y recordatorios.

Con Firebase, el esquema remoto es:

- `users/{uid}` — perfil.
- `users/{uid}/exercise_logs, workout_sessions, meal_choices, chat_messages, article_states, following, engagement`.
- `posts/{postId}` — feed público.
- `authors/{authorId}` — fichas públicas.
- Storage en `users/{uid}/{profile|workouts|plates|posts}/`.

Las reglas de Firestore definen `signedIn` e `isOwner`. Todo lo que cuelga de `users/{uid}` exige que el uid autenticado coincida. `CloudSyncManager` hace pull al iniciar sesión y push en cada escritura de repositorio. Las fotos locales se suben y la URL `https` reemplaza la ruta interna. Sin

el json de servicios, `NoOpCloudStore.isEnabled` es falso y ninguna de esas llamadas sale del teléfono.

`ImageStore.persist` copia el URI elegido por la persona al almacenamiento interno de la app, para que la foto sobreviva aunque el proveedor de contenido original la retire.

8. Sistema de diseño

El tema es oscuro con acento lima `#CBF43F` sobre fondo `#0B0D0C`. Superficies en `#15181B` y `#1E2227`. Texto principal `#F4F6F3`, texto secundario `#8A90A0`. Acentos de apoyo: aqua `#5BC8FF`, naranja `#FF9A4D`, magenta `#FF6FA5`, violeta `#A78BFA` y peligro `#FF6B6B`. Esos mismos tintes separan los macronutrientes.

`FormaTypography` usa una sans del sistema con pesos Black en display (40, 32 y 26 sp) y una escala compacta para cuerpo y labels. El tracking negativo en los títulos grandes es parte de la voz visual: deportiva, corta, con mucho aire negro.

La presentación de este reporte reutiliza esa paleta a propósito, para que explicar el producto se sienta como usar el producto.

9. Stack y pruebas

CAPA	VERSIÓN EN EL CATÁLOGO
Android Gradle Plugin	8.7.3
Kotlin / KSP	2.0.21 / 2.0.21-1.0.28
Compose BOM	2024.12.01
Hilt	2.52, navegación 1.2.0
Navigation Compose	2.8.5
Room	2.6.1
DataStore	1.1.1
Coil / OkHttp	2.7.0 / 4.12.0
Coroutines	1.9.0
Firebase BOM	33.7.0

Serialization

1.7.3

Release activa minify y shrink con las reglas de `proguard-rules.pro`. El version catalog vive en `gradle/libs.versions.toml`. El wrapper trae Gradle 8.11.1.

Pruebas unitarias, 15 casos con nombres en español:

- `RoutineGeneratorTest` — la semana tiene siete días distintos; el mismo perfil produce la misma rutina; cada día de entreno trae ejercicios, minutos y kcal; el gym solo usa equipo marcado; cada deporte genera su propia rutina.
- `MealPlanGeneratorTest` — cinco opciones sin repetir; la selección por defecto se acerca a la meta; el plan cambia entre días; el mismo perfil y día son estables; se priorizan ingredientes que gustan.
- `LocalCoachTest` — la proteína usa el peso real; la respuesta ignora acentos; nunca vuelve vacía; funciona sin perfil; el consejo de proteína cambia si el objetivo es bajar grasa.
- `NoOpCloudStoreTest` — el contrato remoto apagado se cumple.

Comandos: `./gradlew test`, `./gradlew assembleDebug`, `./gradlew installDebug`. Si el SDK no está en la ruta por defecto, `local.properties` lleva `sdk.dir`.

10. Decisiones y límites

Decisiones que conviene defender

- **Determinismo antes que un modelo generativo opaco.** La rutina y la dieta salen de catálogos reales. Se pueden repetir en una prueba y se pueden explicar en una diapositiva.
- **Una interfaz de IA.** La UI nunca importa `OkHttp` ni el coach. Cambiar de proveedor es un binding.
- **Room como fuente de la interfaz.** La nube es una réplica. Quitar Firebase no abre un segundo producto.
- **Comunidad sin comentarios.** El reconocimiento cabe en cinco reacciones. El modelo no reserva un hilo.
- **Equipo como filtro duro.** Un ejercicio de gym cuyo `requires` no está marcado no entra en la sesión.
- **Español en el contenido, no solo en los strings.** Artículos, tips, recetas y respuestas del coach están escritos en el idioma de la interfaz.

Límites actuales

- El chat remoto no genera todavía rutinas ni menús en JSON. Esas dos salidas siguen en el motor local a propósito, para no aceptar un ejercicio inventado.
- La migración de Room es destructiva. El esquema está en la versión 1.
- El IMC y las calorías son orientaciones de producto. El coach lo dice cuando hablan de IMC: no distingue músculo de grasa y no diagnostica.
- Las notificaciones de Inicio son un contador derivado, no un canal push.
- La semilla de comunidad es contenido de demostración para que el feed no nazca vacío.

Cómo encender lo opcional

Firebase: colocar `app/google-services.json`, activar Auth, Firestore y Storage, y desplegar `firebase/firestore.rules` junto con `firebase/storage.rules`. IA remota: definir las tres propiedades `FORMA_AI_*` y cambiar el binding de `AiModule` a `RemoteAiRepository`.

Cierre

FORMA es una función del perfil, dibujada como una app. El perfil entra por cinco pasos, se guarda en Room, alimenta tres motores deterministas y se reparte en seis pestañas. Cada gesto de la interfaz tiene una función con nombre. Este reporte es el índice de esos nombres.

Acompaña a la [presentación](#) y al [guión](#). Cifras contadas sobre el código de la versión 1.0.0: 8 deportes, 60 ejercicios, 57 recetas, 63 ingredientes, 117 piezas de equipo, 10 artículos, 8 autores semilla, 16 intenciones del coach, 76 archivos Kotlin.